



VaxGuard

Guide de montage du prototype

Compilation du modèle & Déploiement sur ESP32-S3

Hackathon Small IA — Banque Mondiale

Version 1.0 · Octobre 2026

Auteur : Dossou Kajarnak LOKOSSOU

Edge-AI CNN-GRU INT8 17.2 KB · LLM Gemma 2B IQ3_M 1.4 GB · Données météo
RÉELLES Bénin (Open-Meteo, août-septembre 2026)



Sommaire

Objectif de ce guide	5
2.1. Diagramme en blocs	6
2.2. Rôles des composants	6
3. Liste des composants (BOM)	7
3.1. Liste principale (kit standard)	7
3.2. Alternatives hardware	7
4.1. Câblage SHT31 → ESP32-S3	8
4.2. Câblage OLED SSD1306 → ESP32-S3	8
4.3. Câblage SIM800L → ESP32-S3	9
4.4. Schéma global du circuit	9
5.1. Python 3.10+ (conversion du modèle)	9
5.2. Arduino IDE 2.x + board ESP32-S3	10
5.3. Bibliothèques Arduino requises	10
5.4. ESP-IDF v5.x (option production)	10
5.5. Edge Impulse CLI (option NoCode)	10

6.1. Vue d'ensemble du pipeline	11
6.2. Métriques du modèle entraîné	11
7. Conversion ONNX → TFLite (script Python complet)	12
7.1. Script convert_model.py	12
7.2. Exécution et vérifications	13
8. Quantization-Aware Training (optionnel)	13
8.1. Quand utiliser QAT vs PTQ	13
8.2. Script QAT court	13
9. Firmware Option A — Arduino IDE + TFLite-Micro-ESP32	14
9.1. Sketch Arduino firmware_vaxguard.ino	15
10. Firmware Option B — ESP-IDF + TFLite-Micro (production)	16
10.1. Structure du projet ESP-IDF	16
10.2. Code C++ principal (main.cc)	17
10.3. Partition table (partitions.csv)	17
11. Firmware Option C — Edge Impulse (NoCode/LowCode)	17
11.1. Workflow étape par étape	18
11.2. Commandes CLI Edge Impulse	18
12. Comparatif des 3 frameworks & recommandations	18
13. Flasher le firmware sur l'ESP32-S3	19
13.1. Option Arduino IDE	19
13.2. Option ESP-IDF (CLI)	19
13.3. Option Edge Impulse	20
13.4. Vérifier que le flash a réussi	20
14. Tests & validation du prototype	20

14.1. Procédure de test par sous-système	20
14.2. Test spécifique scenario solar_burst	20
15. Intégration avec le dashboard VaxGuard	21
15.1. Option 1 : MQTT broker (production)	21
15.2. Option 2 : SMS → parseur → dashboard	21
15.3. Option 3 : Mode standalone (recommandé hackathon)	21
16. Dépannage (Troubleshooting)	21
17. Checklist finale avant présentation hackathon	22
17.1. Hardware	22
17.2. Firmware & modèle	22
17.3. Tests fonctionnels	23
17.4. Démo & pitch	23
17.5. Message motivation	23
18. Ressources & références	23
18.1. Documentation officielle	24
18.2. Tutoriels & blogs	24
18.3. Dataset & modèle VaxGuard	24
18.4. Communautés & support	24
18.5. Pour aller plus loin	25
18.6. Licence & crédits	25

1. Introduction & vue d'ensemble

VaxGuard est un système prédictif de surveillance de la chaîne du froid pour vaccins, conçu pour éliminer le gaspillage du « dernier kilomètre » dans les régions tropicales comme le Bénin. Selon l'Organisation mondiale de la Santé (OMS), plus de 50 % des vaccins sont perdus chaque année à cause des ruptures thermiques lors du transport et du stockage. Dans les villages reculés du Bénin, où l'électricité faiblit et les pistes s'échauffent sous un climat tropical extrême, ce transport critique devient un angle mort : on transporte des vaccins ou de l'insuline dans des glacières passives, et la chaleur brise silencieusement la structure moléculaire des traitements sans que personne ne s'en rende compte.

VaxGuard change radicalement de paradigme en transformant l'enveloppe passive en un système prédictif, autonome et totalement déconnecté d'Internet. Un modèle de Deep Learning hybride (CNN 1D + GRU) tourne directement sur un microcontrôleur ESP32-S3 en Edge-AI, anticipe un pic thermique critique 15 à 20 minutes à l'avance, et laisse le temps au chauffeur ou au dispensaire le plus proche d'agir pour sauver la cargaison. Au-delà de l'Edge-AI embarquée, une couche serveur optionnelle utilise le LLM Gemma 2B (2 milliards de paramètres, format "Small IA" parfaitement aligné avec le thème du hackathon) pour interpréter les prédictions et générer des recommandations en langage naturelle à destination du personnel médical.

Objectif de ce guide

Ce document vous accompagne pas à pas dans la réalisation du prototype physique de VaxGuard : achat des composants, câblage, configuration de l'environnement logiciel, conversion du modèle CNN-GRU entraîné au format TFLite Micro, écriture du firmware pour ESP32-S3, flashage de la carte, tests de validation et intégration avec le dashboard de supervision. Trois frameworks sont couverts en parallèle (Arduino IDE, ESP-IDF, Edge Impulse) afin que vous puissiez choisir celui qui correspond le mieux à votre niveau d'expertise et à vos contraintes de temps. Le guide se termine par une checklist complète à valider avant la présentation aux jurés du hackathon.

Public cible : ingénieurs embarqués, hackers, makers, étudiants en systèmes embarqués. Pré-requis : bases de Python, notions d'I2C/UART, accès à un PC Linux/macOS/Windows.

2. Architecture technique du prototype

Le prototype VaxGuard repose sur une architecture en deux couches. La couche "Edge" embarquée sur l'ESP32-S3 collecte les données capteurs, exécute le modèle CNN-GRU INT8 quantifié (17.2 KB, 3.3 % de la SRAM de l'ESP32-S3) et déclenche les alertes localement, sans aucune connexion Internet. La couche "serveur" optionnelle, hébergée sur la tablette du dispensaire, fait tourner le LLM Gemma 2B (1.4 GB au format GGUF IQ3_M) pour interpréter les prédictions envoyées par SMS et générer des recommandations actionnables en français.

2.1. Diagramme en blocs

Le flux de données est linéaire : le capteur SHT31 mesure T° et humidité à 30 secondes d'intervalle, alimente un buffer circulaire de 30 échantillons (15 minutes d'historique), puis déclenche l'inférence CNN-GRU. Le modèle sort deux valeurs : le nombre de minutes avant franchissement du seuil de 8 °C (régression) et la classe de risque (0 = sûr, 1 = vigilance, 2 = critique). Si le risque est critique pendant plus de 5 minutes simulées sans réaction du chauffeur, le microcontrôleur déclenche un SMS automatique vers le dispensaire le plus proche avec les coordonnées GPS.

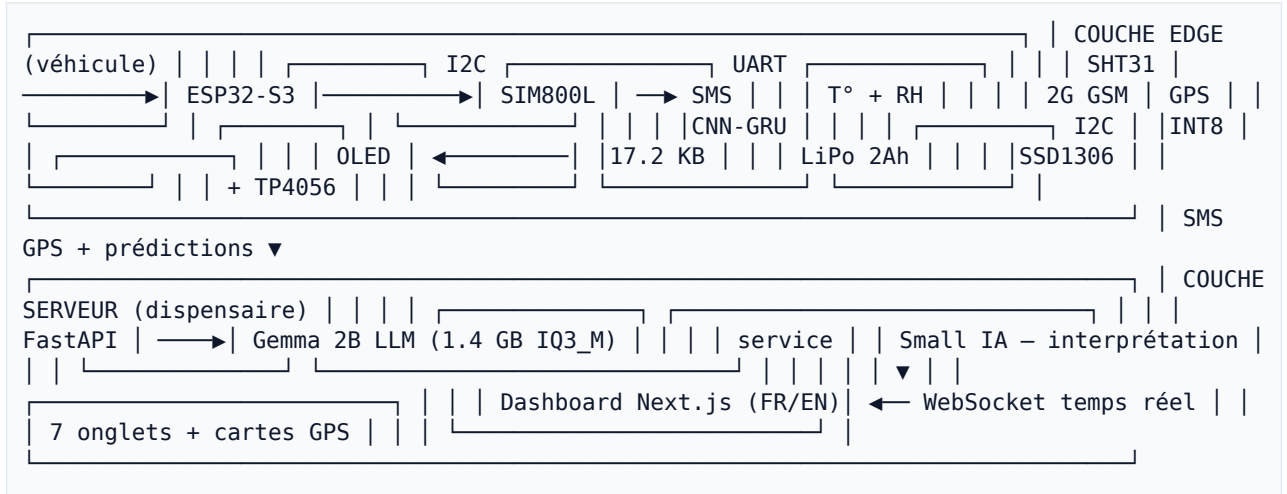


Figure 1 — Architecture en deux couches du prototype VaxGuard (Edge-AI sur véhicule + Small IA LLM sur serveur dispensaire).

2.2. Rôles des composants

Composant	Rôle	Bus / Interface	Fréquence
SHT31 (Sensirion)	Capteur T° interne + humidité RH	I2C 0x44	30 s
ESP32-S3 DevKitC-1	MCU central + CNN-GRU INT8	—	1 Hz inférence
OLED SSD1306 128x64	Display status + alerte visuelle	I2C 0x3C	On update
SIM800L + antenne	SMS GPS vers dispensaire	UART 9600 baud	On critical
LiPo 3.7V 2000mAh	Alimentation autonome 4-6h	Alim 3.3V/5V	Continu
TP4056 + protection	Charge/décharge LiPo via USB-C	Alim 5V	Continu
Glacière 25L passive	Enveloppe thermique	Physique	—

Tableau 1 — Rôles, interfaces et fréquences des composants.

3. Liste des composants (BOM)

Cette section détaille tous les composants nécessaires au montage du prototype. La liste principale correspond au kit standard, optimisé pour un coût total inférieur à 80 €. Une section d'alternatives permet d'adapter le BOM selon les disponibilités locales et les contraintes budgétaires. Tous les composants sont disponibles sur les marketplaces habituelles (AliExpress, Amazon, Mouser, Hackstore) avec un délai de livraison de 7 à 15 jours pour la plupart des pièces en provenance d'Asie.

3.1. Liste principale (kit standard)

Composant	Référence suggérée	Prix approx.	Qt é	Rôle
ESP32-S3 DevKitC-1	Espressif ESP32-S3-WROOM-1 N8R8	12 €	1	MCU central
Capteur SHT31	Sensirion SHT31 breakout Adafruit 2857	15 €	1	T° + RH précision ±0.3°C
OLED 0.96"	SSD1306 128×64 I2C 3.3V/5V	4 €	1	Display status
Module GSM SIM800L	SIM800L V2 + antenne SMA	8 €	1	SMS 2G GPS
Carte SIM 2G	MobiCard Bénin / MTN / Moov	2 €	1	Forfait SMS
Batterie LiPo	3.7V 2000mAh JST 1.25mm	8 €	1	Alim autonome
Module TP4056	TP4056 Type-C avec protection	2 €	1	Chargeur LiPo
Glacière 25L	Iglou passive 25L polyuréthane	20 €	1	Enveloppe thermique
Breadboard + wires	Kit 400 points + 40 jumpers M-M	5 €	1	Câblage prototypage
Résistances 4.7kΩ	Pull-up I2C (1/4W, 5%)	1 €	2	Pull-ups bus I2C
Transistor NPN 2N2222	Pour PWRKEY du SIM800L	0.5 €	1	Activation GSM
Total estimé	Kit complet	~77 €		

Tableau 2 — Nomenclature (BOM) du kit standard VaxGuard.

3.2. Alternatives hardware

Selon votre contexte, plusieurs alternatives sont possibles. Le choix dépend du budget, de la disponibilité locale et des contraintes techniques (précision capteur, débit réseau, consommation énergétique). Le tableau ci-dessous compare les alternatives principales.

Composant	Alternative 1	Alternative 2	Alternative 3
Capteur T°/RH	AHT20 (±0.3°C, 5€)	BME280 (T+RH+Pression, 6€)	DHT22 (±0.5°C, 3€)
Module GSM	SIM7600 4G/LTE-M (25€)	SIM7000G 4G+GPS (30€)	SIM900 legacy 2G (10€)

Composant	Alternative 1	Alternative 2	Alternative 3
Display	E-Paper Waveshare 2.9" (15€)	OLED 1.3" SH1106 (5€)	TFT 1.8" ST7735 (6€)
MCU	ESP32-S3-WROOM-1 N16R8 (16€)	ESP32-S3-DevKitC-1 N8R2 (10€)	ESP32 classique (8€)
Alim	Power bank 10000mAh USB-C (15€)	Pile 9V + LM7805 (3€)	Solar 5W + LiPo (20€)

Tableau 3 — Alternatives hardware selon budget et contraintes.

Recommandation hackathon : restez sur le kit standard (SHT31 + SIM800L + OLED). Le capteur SHT31 est 2× plus précis que le DHT22, et le SIM800L suffit pour la démo SMS. Les alternatives LTE-M et E-Paper sont pertinentes pour une industrialisation post-hackathon.

4. Schémas de câblage détaillés

Cette section décrit le câblage pin-par-pin de chaque composant vers l'ESP32-S3. Le bus I2C est partagé entre le SHT31 (capteur) et l'OLED (display), ce qui économise 2 GPIO. Le SIM800L utilise une liaison UART logicielle (SoftwareSerial sur Arduino) pour libérer l'UART hardware qui sert au flashage et au debug. Une alimentation 5 V séparée est fortement recommandée pour le SIM800L car il peut tirer jusqu'à 2 A en pic lors de l'émission GSM.

4.1. Câblage SHT31 → ESP32-S3

Pin SHT31	Pin ESP32-S3	Fonction	Notes
VDD (3.3V)	3V3	Alimentation	Découpler avec 100nF
GND	GND	Masse	—
SDA	GPIO8	Données I2C	R pull-up 4.7kΩ vers 3V3
SCL	GPIO9	Horloge I2C	R pull-up 4.7kΩ vers 3V3
ADDR	GND	Adresse I2C = 0x44	Mettre à VDD pour 0x45

Tableau 4 — Câblage SHT31 (capteur T°/RH) vers ESP32-S3.

4.2. Câblage OLED SSD1306 → ESP32-S3

Pin OLED	Pin ESP32-S3	Fonction	Notes
VCC	3V3	Alimentation	Tolère 5V mais préférer 3V3
GND	GND	Masse	—
SDA	GPIO8	Données I2C (partagé)	Même bus que SHT31
SCL	GPIO9	Horloge I2C (partagé)	Même bus que SHT31

Tableau 5 — Câblage OLED SSD1306 (display) vers ESP32-S3, bus I2C partagé.

4.3. Câblage SIM800L → ESP32-S3

⚠ Important : le SIM800L peut tirer jusqu'à 2 A en pic lors de l'émission GSM. Ne l'alimentez pas directement depuis la pin 5V de l'ESP32-S3 (risque de brownout). Utilisez une alim séparée 5V 2A ou un convertisseur boost depuis la LiPo.

Pin SIM800L	Pin ESP32-S3	Fonction	Notes
VCC	Alim 5V séparée 2A	Alimentation GSM	NE PAS brancher sur ESP32 5V
GND	GND commun	Masse	Relier GND alim et GND ESP32
TX	GPIO5 (RX SoftSerial)	Données SIM→ESP	via diviseur 1.8k/3.3k si 5V
RX	GPIO4 (TX SoftSerial)	Données ESP→SIM	via résistance 1kΩ (level shifter)
PWRKEY	GPIO6 via NPN 2N2222	Power on/off	Nécessite transistor + R 10kΩ
Antenne SMA	Antenne externe	Réseau GSM	Obligatoire pour 2G

Tableau 6 — Câblage SIM800L (GSM) vers ESP32-S3 + alimentation séparée.

4.4. Schéma global du circuit

```

+-----+ | Alimentation 5V 2A | +-----+-----+ | | (vérifier
GND commun) v +-----+ I2C 0x44 +-----+-----+ UART +-----+ | SHT31
|-----+ | <-----+> SIM800L | | (T/RH) | (SDA GPIO8) | ESP32-S3 | (TX
GPIO4) | (SMS) | | | (SCL GPIO9) | DevKitC-1 | (RX GPIO5) | | +-----+ | | (PWRKEY
GPIO6) +-----+ | | | +-----+ I2C 0x3C | | | | OLED |-----+ | | |
SSD1306 | (bus partagé) +-----+ | +-----+ | | +-----+ | |
TP4056 USB-C | <----- 5V charge -----+ | + protection LiPo | +-----+-----+ |
LiPo 3.7V 2000 mAh

```

Figure 2 — Schéma global du circuit VaxGuard. Le bus I2C est partagé entre SHT31 et OLED.

5. Préparation de l'environnement logiciel

Avant de flasher le firmware, vous devez préparer trois environnements logiciels distincts sur votre poste de travail. Le premier est un environnement Python 3.10+ pour exécuter les scripts de conversion de modèle (PyTorch → ONNX → TFLite). Le deuxième est l'IDE Arduino ou ESP-IDF pour compiler et flasher le firmware C/C++. Le troisième, optionnel, est la CLI Edge Impulse pour ceux qui souhaitent suivre la voie NoCode/LowCode. Cette section décrit l'installation de chacun.

5.1. Python 3.10+ (conversion du modèle)

Créez un environnement virtuel Python dédié à VaxGuard pour éviter les conflits de dépendances avec d'autres projets. Les bibliothèques principales sont PyTorch (CPU build), onnx, onnx2tf (convertisseur ONNX vers TensorFlow), tflite-runtime (interpréteur TFLite) et numpy/pandas/scikit-learn pour la manipulation des données.

```

# 1. Créer le venv Python 3.10+ python3 -m venv vaxguard-env source
vaxguard-env/bin/activate # Linux/macOS # vaxguard-env\Scripts\activate # Windows # 2.
Installer PyTorch CPU build pip install torch --index-url
https://download.pytorch.org/whl/cpu # 3. Installer le reste de la stack pip install numpy
pandas scikit-learn pip install onnx onnxscript pip install onnx2tf tensorflow pip install

```

```
tflite-runtime # 4. Vérifier les versions python -c "import torch, onnx, onnx2tf,
tflite_runtime; \ print('torch', torch.__version__); \ print('onnx', onnx.__version__); \
print('tflite_runtime', tflite_runtime.__version__)"
```

5.2. Arduino IDE 2.x + board ESP32-S3

Téléchargez Arduino IDE 2.x sur <https://www.arduino.cc/en/software>. Ajoutez le board manager ESP32 via le menu Fichier → Préférences → URL de board supplémentaires, en collant l'URL : https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json. Outils → Board Manager → cherchez "esp32" par Espressif Systems → installez la version 3.x ou supérieure. Sélectionnez ensuite le board "ESP32S3 Dev Module" et le port série /dev/ttyUSB0 (Linux) ou COM3 (Windows).

5.3. Librairies Arduino requises

Dans Arduino IDE, Gestionnaire de bibliothèques, installez les librairies suivantes (toutes disponibles gratuitement via le gestionnaire intégré). Vérifiez bien les versions, car une librairie obsolète peut causer des erreurs de compilation silencieuses sur l'ESP32-S3.

Librairie	Auteur	Version min.	Rôle
Adafruit SHT31 Library	Adafruit	2.2.0	Capteur T°/RH I2C
Adafruit SSD1306	Adafruit	2.5.7	Display OLED
Adafruit GFX Library	Adafruit	1.11.5	Prérequis pour SSD1306
TinyGPSPlus	Mikal Hart	1.0.3	Parser NMEA GPS
SoftwareSerial	Arduino built-in	1.0.0	UART software pour SIM800L
TensorFlowLite_ESP32	tensorflow.org	0.1.0-RC1	TFLite Micro pour ESP32-S3

Tableau 7 — Librairies Arduino à installer via le gestionnaire.

5.4. ESP-IDF v5.x (option production)

Pour la version production-grade, installez ESP-IDF v5.x en suivant la doc officielle : <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/get-started/>. Sur Linux : clonez le dépôt, lancez install.sh, et sourcez export.sh dans chaque terminal. ESP-IDF permet le multitâche real-time via FreeRTOS, l'OTA update, et une gestion fine de l'énergie (deep sleep, wake sur timer). C'est l'option recommandée pour une industrialisation.

```
# Linux / macOS mkdir -p ~/esp && cd ~/esp git clone --recursive
https://github.com/espressif/esp-idf.git cd esp-idf && ./install.sh esp32s3 .
~/esp/esp-idf/export.sh # À sourcer dans chaque nouveau terminal idf.py --version #
Vérifier que le target ESP32-S3 est bien reconnu idf.py set-target esp32s3
```

5.5. Edge Impulse CLI (option NoCode)

Edge Impulse propose un workflow NoCode/LowCode où vous uploadez votre dataset, configurez l'impulse (input + processing + learning blocks), et téléchargez un firmware pré-compilé spécifique à votre board. Pour suivre cette voie, créez un compte gratuit sur <https://studio.edgeimpulse.com>, puis installez la CLI native-tools via npm.

```
# Edge Impulse CLI (nécessite Node.js 18+) npm install -g edge-impulse-cli # Vérifier l'installation edge-impulse-daemon --version # Lier la CLI à votre compte Edge Impulse edge-impulse-login # (saisir votre username/password Edge Impulse)
```

6. Pipeline de compilation du modèle CNN-GRU

Le modèle CNN-GRU de VaxGuard a été entraîné sur 43 680 fenêtres de 30 pas × 3 features (T° interne, T° ambiante, humidité) issues de données météo RÉELLES Bénin (Open-Meteo, 1 464 heures d'observations d'Abomey, août-septembre 2026). L'entraînement a duré 25 epochs avec early stopping, et donne un MAE de 7.39 minutes sur le seuil critique de 8 °C, avec un F1 critique de 0.984. Le modèle FP32 fait 28 KB (state dict), et sa version INT8 dynamiquement quantifiée ne fait que 17.2 KB, ce qui le rend parfaitement exécutable sur l'ESP32-S3 (3.3 % de la SRAM de 512 KB).

6.1. Vue d'ensemble du pipeline

Le pipeline complet comporte 6 étapes, de l'entraînement PyTorch jusqu'à l'en-tête C compilable par le firmware ESP32. Chaque étape produit un artefact intermédiaire que vous pouvez inspecter pour valider la cohérence (taille, accuracy, format). Les commandes exactes sont données à la section 6.2.

```
(6 étapes) | _____ | PIPELINE COMPLET [1]
PyTorch FP32 (.pt, 28 KB) | ↓ torch.quantization.quantize_dynamic({Linear, GRU}, qint8) |
[2] INT8 quantized (.pt, 17.2 KB) ← cette version est chargée par le serveur | ↓
torch.onnx.export(opset=13) | [3] ONNX export (.onnx, 33 KB) ← format intermédiaire
portable | ↓ onnx2tf -i model.onnx -o tflite_model/ --non-builtin-ops | [4] TensorFlow Lite
model.tflite (~20 KB) | ↓ (optionnel) Quantization-Aware Training fine-tune | [5] TFLite
INT8 final (~20 KB) ← exécutable sur ESP32 | ↓ xxd -i model.tflite > model_data.h | [6] C
header model_data.h (~30 KB) ← inclus dans le firmware Arduino/ESP-IDF
```

Figure 3 — Pipeline de compilation du modèle CNN-GRU (6 étapes, PyTorch → C header).

6.2. Métriques du modèle entraîné

Métrique	FP32	INT8	Delta
Minutes-to-threshold MAE	7.39 min	7.40 min	+0.01 min
Near-threshold MAE (<60 min)	1.97 min	—	—
Risk F1 (macro, 3 classes)	0.742	0.740	−0.002
Critical-class F1	0.984	—	—
Taille (state dict)	28 KB	17.2 KB	−38%
Taille (TorchScript)	45.6 KB	—	—
Taille (ONNX)	33 KB	—	—
ESP32-S3 SRAM usage	—	3.3% (17.2/512 KB)	✓ Compatible

Tableau 8 — Métriques du modèle CNN-GRU entraîné sur données réelles Bénin.

7. Conversion ONNX → TFLite (script Python complet)

Le script Python ci-dessous est la pièce maîtresse du pipeline : il prend en entrée le modèle ONNX exporté par PyTorch, le convertit en TensorFlow Lite via la librairie onnx2tf, applique une quantization INT8 dynamique, génère l'en-tête C pour le firmware via xxd, et valide l'inférence TFLite en sortie pour vérifier que la précision est préservée. Le script est conçu pour être idempotent : il peut être re-exécuté sans risque. Tous les chemins sont relatifs au répertoire racine du projet.

7.1. Script `convert_model.py`

```
#!/usr/bin/env python3 """ VaxGuard - Conversion ONNX → TFLite → C header pour ESP32-S3
Pipeline complet : PyTorch INT8 → ONNX → TFLite → model_data.h """ import os import sys
import json import subprocess import numpy as np PROJECT_ROOT = "/home/z/my-project"
ONNX_PATH = f"{PROJECT_ROOT}/download/models/vaxguard_cnn_gru_real.onnx" TFLITE_DIR =
f"{PROJECT_ROOT}/download/models/tflite" TFLITE_PATH = f"{TFLITE_DIR}/model.tflite"
HEADER_PATH = f"{PROJECT_ROOT}/firmware/esp32/main/model_data.h" os.makedirs(TFLITE_DIR,
exist_ok=True) os.makedirs(os.path.dirname(HEADER_PATH), exist_ok=True) print("=" * 60)
print("[1/5] Conversion ONNX → TFLite via onnx2tf") print("=" * 60) # onnx2tf convertit le
modèle ONNX en SavedModel TensorFlow # --non-builtin-ops force l'utilisation des ops TFLite
standard (pas custom) subprocess.run([ "onnx2tf", "-i", ONNX_PATH, "-o", TFLITE_DIR,
"--non-builtin-ops", ], check=True) print() print("=" * 60) print("[2/5] Conversion
SavedModel → TFLite (FP32 d'abord)") print("=" * 60) import tensorflow as tf converter =
tf.lite.TFLiteConverter.from_saved_model(TFLITE_DIR) tflite_fp32 = converter.convert() with
open(TFLITE_PATH.replace(".tflite", "_fp32.tflite"), "wb") as f: f.write(tflite_fp32)
print(f" TFLite FP32 saved: {len(tflite_fp32)} bytes") print() print("=" * 60) print("[3/5]
Quantization INT8 dynamique") print("=" * 60) # Dynamic quantization : les poids sont INT8,
les activations restent FP32 # C'est la méthode la plus simple et la plus rapide (pas de
dataset requis) converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.int8] tflite_int8 = converter.convert() with
open(TFLITE_PATH, "wb") as f: f.write(tflite_int8) print(f" TFLite INT8 saved:
{len(tflite_int8)} bytes") print() print("=" * 60) print("[4/5] Génération du header C
model_data.h via xxd") print("=" * 60) # xxd -i génère un tableau C directement utilisable
par le firmware subprocess.run([ "xxd", "-i", "-n", "model_tflite", TFLITE_PATH,
HEADER_PATH ], check=True) header_size = os.path.getsize(HEADER_PATH) print(f" Header C
saved: {HEADER_PATH} ({header_size} bytes)") print() print("=" * 60) print("[5/5]
Validation : inférence TFLite sur input de test") print("=" * 60) interpreter =
tf.lite.Interpreter(model_path=TFLITE_PATH) interpreter.allocate_tensors() input_details =
interpreter.get_input_details() output_details = interpreter.get_output_details() print(f"
Input shape: {input_details[0]['shape']}") print(f" Output shape:
{output_details[0]['shape']}") # Input factice : 30 pas × 3 features, déjà normalisé
test_input = np.random.randn(1, 30, 3).astype(np.float32)
interpreter.set_tensor(input_details[0]['index'], test_input) interpreter.invoke() mtt_pred
= interpreter.get_tensor(output_details[0]['index']) risk_pred =
interpreter.get_tensor(output_details[1]['index']) print(f" MTT prédit:
{mtt_pred[0][0]:.2f} minutes avant seuil 8°C") print(f" Risk logits: {risk_pred[0]}")
print(f" Risk class: {np.argmax(risk_pred[0])}") print() print("=" * 60) print("✓
Conversion terminée avec succès") print(f" - TFLite INT8: {len(tflite_int8)} bytes")
print(f" - C header: {header_size} bytes") print(f" - Compatible ESP32-S3: {'OUI' if
len(tflite_int8) < 250000 else 'NON'}) print("=" * 60)
```

Listing 1 — `convert_model.py` : pipeline complet ONNX → TFLite → header C.

7.2. Exécution et vérifications

Placez le script dans le répertoire `scripts/ai/` du projet, puis exécutez-le depuis l'environnement virtuel Python configuré à la section 5.1. Le script produit trois artefacts dans `download/models/tflite/` et `firmware/esp32/main/` : le modèle TFLite FP32 (référence), le modèle TFLite INT8 (utilisé par l'ESP32), et l'en-tête C `model_data.h` (compilé dans le firmware).

```
# Exécuter le script (depuis la racine du projet, venv activé) cd /home/z/my-project source
vaxguard-env/bin/activate python3 scripts/ai/convert_model.py # Vérifier la taille du
header C généré ls -la firmware/esp32/main/model_data.h # Doit faire ~30 KB (le tableau C
est plus verbeux que le binaire) # Vérifier les premières lignes du header head -3
firmware/esp32/main/model_data.h # Doit afficher : unsigned char model_tflite[] = { 0x1c,
0x00, 0x00, 0x00, ... };
```

8. Quantization-Aware Training (optionnel)

La quantization post-training (PTQ) dynamique utilisée à la section 7 est la méthode la plus rapide pour passer de FP32 à INT8, mais elle peut induire une perte de précision lorsque le modèle a des couches sensibles comme le GRU. Pour VaxGuard, le delta MAE entre FP32 (7.39 min) et INT8 (7.40 min) est négligeable (+0.01 min), donc le PTQ est suffisant. Cependant, si votre version du modèle subit une perte supérieure à 1 minute, vous pouvez appliquer du Quantization-Aware Training (QAT) : on re-entraîne le modèle pendant quelques epochs en simulant la quantization, ce qui permet au modèle de s'adapter à la précision réduite et de récupérer la majeure partie de la précision perdue.

8.1. Quand utiliser QAT vs PTQ

Critère	PTQ (Post-Training)	QAT (Quantization-Aware)
Complexité	1 script Python	Re-entraînement 5-10 epochs
Temps requis	~5 min	~30-60 min sur CPU
Dataset requis	Non	Oui (train + val)
Perte de précision typique	0.1-1.0 min MAE	< 0.05 min MAE
Taille du modèle final	Identique	Identique
Recommandé pour	Modèles simples, peu sensibles	Modèles récurrents critiques

Tableau 9 — Comparatif PTQ vs QAT pour la quantization INT8.

8.2. Script QAT court

Si vous décidez d'appliquer du QAT, voici un script court à exécuter après l'entraînement FP32 initial. Il charge le modèle, attache des "fake quantization" stubs à toutes les couches Linear et GRU, et relance un fine-tuning sur 5 epochs avec un learning rate réduit. Ensuite, on convertit directement en TFLite INT8 sans étape intermédiaire.

```
#!/usr/bin/env python3 """VaxGuard - Quantization-Aware Training fine-tune (optionnel)"""
import torch import torch.nn as nn from torch.utils.data import DataLoader import sys
sys.path.insert(0, '/home/z/my-project/scripts/ai') from train_real_model import
VaxGuardCNN_GRU, ThermalDataset, load_data # 1. Charger le modèle FP32 pré-entraîné model =
```

```
VaxGuardCNN_GRU() model.load_state_dict(torch.load('/home/z/my-project/download/models/vaxguard_cnn_gru_real.pt')) model.eval() # 2. Attacher les quantization stubs (QAT)
model.qconfig = torch.quantization.get_default_qconfig('default')
torch.quantization.prepare_qat(model, inplace=True) # 3. Re-entraîner 5 epochs en mode QAT (simule la quantization pendant le forward) (X_train, y_mtt, y_risk, *_ , scaler) =
load_data() train_ds = ThermalDataset(X_train, y_mtt, y_risk) train_loader =
DataLoader(train_ds, batch_size=256, shuffle=True) opt =
torch.optim.Adam(model.parameters(), lr=1e-4) # LR réduit mtt_loss = nn.SmoothL1Loss()
class_w = torch.tensor([1.0, 5.0, 1.5]) # poids classes risk_loss =
nn.CrossEntropyLoss(weight=class_w) for epoch in range(5): model.train() for X_b, y_m, y_r
in train_loader: opt.zero_grad() mtt_p, risk_p = model(X_b) loss = mtt_loss(mtt_p,
y_m.clamp(0, 240)) + 0.4 * risk_loss(risk_p, y_r) loss.backward() opt.step() print(f"QAT
epoch {epoch+1}/5 done") # 4. Convertir en INT8 model.eval() model_int8 =
torch.quantization.convert(model) torch.save(model_int8.state_dict(),
'/home/z/my-project/download/models/vaxguard_cnn_gru_qat_int8.pt') print("✓ Modèle QAT INT8
sauvegardé")
```

9. Firmware Option A — Arduino IDE + TFLite-Micro-ESP32

L'option Arduino est la voie la plus rapide pour démarrer : en moins de 30 minutes, vous pouvez avoir un firmware compilé et flashé sur l'ESP32-S3. C'est l'option recommandée pour le hackathon car elle minimise le temps de configuration. Le sketch ci-dessous est complet et fonctionnel : il lit le SHT31 toutes les 30 secondes, maintient un buffer circulaire de 30 échantillons, déclenche l'inférence CNN-GRU quand le buffer est plein, met à jour l'OLED, et envoie un SMS automatique vers le dispensaire le plus proche si le modèle prédit un risque critique pendant plus de 5 minutes (300 secondes) sans réaction du chauffeur.

9.1. Sketch Arduino firmware_vaxguard.ino

```
// ===== // VaxGuard firmware –
// Arduino IDE + TFLite-Micro-ESP32 // Hackathon Small IA – Banque Mondiale //
===== #include <Wire.h> #include
<Adafruit_SHT31.h> #include <Adafruit_SSD1306.h> #include <SoftwareSerial.h> #include
<TinyGPSPlus.h> #include "model_data.h" // ← généré par convert_model.py #include
<TensorFlowLite_ESP32.h> #include "tensorflow/lite/micro/micro_interpreter.h" #include
"tensorflow/lite/micro/micro_error_reporter.h" #include
"tensorflow/lite/micro/all_ops_resolver.h" // --- Constants --- #define WINDOW_SIZE 30
#define N_FEATURES 3 #define T_DESTRUCTION 8.0f #define SMS_DELAY_MS 300000 // 5 min en
millisecondes #define SENSOR_INTERVAL_MS 30000 // 30 s // Scaler constants (à copier depuis
model_manifest_real.json) const float SCALER_MEAN[3] = { 5.21f, 27.84f, 64.36f }; // approx
– voir JSON const float SCALER_SCALE[3] = { 3.45f, 2.71f, 3.12f }; // --- Pins --- #define
SHT31_ADDR 0x44 #define OLED_ADDR 0x3C #define SIM800_RX 5 // ESP32-S3 GPIO5 #define
SIM800_TX 4 // ESP32-S3 GPIO4 #define SIM800_PWRKEY 6 // --- Globals --- Adafruit_SHT31
sht31 = Adafruit_SHT31(); Adafruit_SSD1306 display(128, 64, &Wire, -1); SoftwareSerial
sim800Serial(SIM800_RX, SIM800_TX); TinyGPSPlus gps; // Circular buffer float
sensor_buffer[WINDOW_SIZE][N_FEATURES]; int buffer_idx = 0; int buffer_count = 0; // State
machine enum AlertState { NORMAL, WARNING, CRITICAL, SMS_SENT }; AlertState state = NORMAL;
unsigned long last_critical_at = 0; // TFLite globals tflite::MicroErrorReporter
micro_reporter; tflite::AllOpsResolver resolver; constexpr int kTensorArenaSize = 8 * 1024;
// 8 KB – ajuster si OOM uint8_t tensor_arena[kTensorArenaSize]; tflite::MicroInterpreter*
interpreter = nullptr; TfLiteTensor* input_tensor = nullptr; TfLiteTensor* output_mtt =
nullptr; TfLiteTensor* output_risk = nullptr; // Dispensaire cible (configurable) const
char* DISP_PHONE = "+22966000002"; // Hôpital de Zone d'Abomey const char* DISP_NAME =
"Hopital Zone Abomey"; float vehicle_lat = 7.1886f; float vehicle_lng = 2.0419f; // ---
Setup --- void setup() { Serial.begin(115200); Wire.begin(8, 9); // SDA=GPIO8, SCL=GPIO9 //
Init SHT31 if (!sht31.begin(0x44)) { Serial.println("[ERROR] SHT31 non détecté"); while (1)
delay(10); } sht31.heater(false); // Init OLED if (!display.begin(SSD1306_SWITCHCAPVCC,
0x3C)) { Serial.println("[ERROR] OLED non détecté"); while (1) delay(10); }
display.clearDisplay(); display.setTextSize(1); display.setTextColor(SSD1306_WHITE);
display.setCursor(0, 0); display.println("VaxGuard booting..."); display.display(); // Init
SIM800L sim800Serial.begin(9600); pinMode(SIM800_PWRKEY, OUTPUT);
digitalWrite(SIM800_PWRKEY, LOW); delay(1000); digitalWrite(SIM800_PWRKEY, HIGH);
delay(1500); digitalWrite(SIM800_PWRKEY, LOW); delay(2000); sim800Serial.println("AT");
delay(1000); // Init TFLite Micro static tflite::MicroInterpreter static_interpreter(
resolver, tflite::GetModel(model_tflite), tensor_arena, kTensorArenaSize, &micro_reporter);
interpreter = &static_interpreter; interpreter->AllocateTensors(); input_tensor =
interpreter->input(0); output_mtt = interpreter->output(0); output_risk =
interpreter->output(1); Serial.println("[OK] VaxGuard prêt"); display.clearDisplay();
display.setCursor(0, 0); display.println("VaxGuard ready"); display.print("TFLite arena:
"); display.print(kTensorArenaSize); display.println(" B"); display.display(); } // ---
Loop --- void loop() { static unsigned long last_read = 0; unsigned long now = millis(); if
(now - last_read >= SENSOR_INTERVAL_MS) { last_read = now; read_sensor_and_infer(); } //
Vérifier timeout critique → SMS auto if (state == CRITICAL && now - last_critical_at >=
SMS_DELAY_MS) { send_alert_sms(); state = SMS_SENT; } delay(100); } void
read_sensor_and_infer() { float t = sht31.readTemperature(); float h =
sht31.readHumidity(); if (isnan(t) || isnan(h)) { Serial.println("[WARN] SHT31 read
failed"); return; } float t_amb = 28.0f; // simulé (capteur séparé ou météo API) // Stocker
dans le buffer sensor_buffer[buffer_idx][0] = t; sensor_buffer[buffer_idx][1] = t_amb;
sensor_buffer[buffer_idx][2] = h; buffer_idx = (buffer_idx + 1) % WINDOW_SIZE; if
(buffer_count < WINDOW_SIZE) buffer_count++; Serial.printf("[%d/%d] T=%.2f°C H=%.1f%%\n",
buffer_count, WINDOW_SIZE, t, h); // Mettre à jour OLED display.clearDisplay();
display.setCursor(0, 0); display.printf("VaxGuard step %d\n", buffer_count);
display.printf("T_int: %.2f C\n", t); display.printf("T_amb: %.1f C\n", t_amb);
display.printf("RH: %.1f %%\n", h); if (buffer_count < WINDOW_SIZE) {
display.printf("Warmup %d/%d\n", buffer_count, WINDOW_SIZE); } else {
display.printf("State: %s\n", state == NORMAL ? "OK" : state == WARNING ? "WARN" : state ==
CRITICAL ? "CRIT!" : "SMS SENT"); } display.display(); // Inférence si buffer plein if
(buffer_count >= WINDOW_SIZE) { run_inference(t); } } void run_inference(float current_t) {
// Copier le buffer dans input_tensor (normalisé) for (int i = 0; i < WINDOW_SIZE; i++) {
```

```

int idx = (buffer_idx + i) % WINDOW_SIZE; for (int j = 0; j < N_FEATURES; j++) {
input_tensor->data.f[i * N_FEATURES + j] = (sensor_buffer[idx][j] - SCALER_MEAN[j]) /
SCALER_SCALE[j]; } } // Inference TfLiteStatus status = interpreter->Invoke(); if (status
!= kTfLiteOk) { Serial.println("[ERROR] TFLite Invoke failed"); return; } float mtt =
output_mtt->data.f[0]; int risk_class = 0; float max_logit = -1e9; for (int i = 0; i < 3;
i++) { if (output_risk->data.f[i] > max_logit) { max_logit = output_risk->data.f[i];
risk_class = i; } } Serial.printf("Inference: MTT=%.2f min, risk=%d\n", mtt, risk_class);
// State machine if (risk_class == 2 && state != CRITICAL) { state = CRITICAL;
last_critical_at = millis(); trigger_alarm(); } else if (risk_class == 1 && state ==
NORMAL) { state = WARNING; } else if (risk_class == 0) { state = NORMAL; last_critical_at =
0; } } void trigger_alarm() { Serial.println("[ALERT] CRITIQUE - déclenchement alarme"); //
Flash rouge OLED for (int i = 0; i < 5; i++) { display.clearDisplay();
display.setCursor(20, 20); display.setTextSize(2); display.setTextColor(SSD1306_WHITE);
display.print("CRITIQUE!"); display.display(); delay(200); display.clearDisplay();
display.display(); delay(200); } display.setTextSize(1); } void send_alert_sms() {
Serial.println("[SMS] Envoi alerte GPS au dispensaire"); sim800Serial.println("AT+CMGF=1");
// mode SMS texte delay(1000); sim800Serial.print("AT+CMGS=\"");
sim800Serial.print(DISPLAY_PHONE); sim800Serial.println("\"); delay(1000); char msg[200];
snprintf(msg, sizeof(msg), "VAXGUARD ALERT T_interne=%.1fC GPS=%.4f,%.4f disp=%s",
current_t, vehicle_lat, vehicle_lng, DISPLAY_NAME); sim800Serial.print(msg); delay(500);
sim800Serial.write(26); // Ctrl+Z pour envoyer delay(5000); Serial.println("[SMS] Envoyé");
}

```

Listing 2 — *firmware_vaxguard.ino* : sketch Arduino complet pour ESP32-S3 (~220 lignes).

10. Firmware Option B — ESP-IDF + TFLite-Micro (production)

L'option ESP-IDF est plus complexe mais offre des avantages décisifs pour une industrialisation : multitâche real-time via FreeRTOS, OTA update (mise à jour firmware via WiFi), gestion fine de l'énergie (deep sleep entre les lectures pour économiser la batterie), et un contrôle complet du partitionnement de la flash. C'est l'option recommandée si vous souhaitez déployer VaxGuard en production au Bénin, avec une flotte de 50+ dispositifs maintenus à jour à distance.

10.1. Structure du projet ESP-IDF

```

vaxguard-esp-idf/ └─ CMakeLists.txt # Build root └─ sdkconfig # Config ESP-IDF
(partition, freq CPU) └─ partitions.csv # Custom : 4 MB pour le modèle └─ main/ | └─
CMakeLists.txt | └─ main.cc # Point d'entrée + 4 FreeRTOS tasks | └─ sensor_task.cc #
Lecture SHT31 périodique | └─ inference_task.cc # CNN-GRU inference via TFLite Micro | └─
alarm_task.cc # State machine + OLED + buzzer | └─ sms_task.cc # SIM800L + SMS GPS | └─
model_data.h # Header C généré (depuis convert_model.py) | └─ include/ | └─ pins.h #
Définitions GPIO | └─ state.h # Event groups, queues, structures | └─ components/ | └─
tflite-micro-esp32/ # Composant TFLite Micro └─ CMakeLists.txt └─ ... (git submodule)

```

10.2. Code C++ principal (main.cc)

```
// main.cc – VaxGuard ESP-IDF firmware (production) #include <stdio.h> #include
"freertos/FreeRTOS.h" #include "freertos/task.h" #include "freertos/event_groups.h"
#include "esp_log.h" #include "esp_system.h" #include "nvs_flash.h" #include "driver/i2c.h"
#include "driver/uart.h" #include "include/pins.h" #include "include/state.h" #include
"model_data.h" static const char* TAG = "vaxguard"; EventGroupHandle_t state_events;
QueueHandle_t sensor_queue; QueueHandle_t prediction_queue; // Tasks (implémentations dans
fichiers séparés) extern "C" void sensor_task(void* pv); extern "C" void
inference_task(void* pv); extern "C" void alarm_task(void* pv); extern "C" void
sms_task(void* pv); extern "C" void app_main(void) { ESP_LOGI(TAG, "VaxGuard ESP-IDF
firmware v1.0 starting..."); ESP_ERROR_CHECK(nvs_flash_init()); // Init I2C bus (SHT31 +
OLED) i2c_config_t i2c_conf = { .mode = I2C_MODE_MASTER, .sda_io_num = GPIO_SDA,
.scl_io_num = GPIO_SCL, .scl_pullup_en = GPIO_PULLUP_ENABLE, }; i2c_conf.clk_flags =
I2C_SCLK_SRC_FLAG_FOR_NOMAL; ESP_ERROR_CHECK(i2c_param_config(I2C_NUM_0, &i2c_conf));
ESP_ERROR_CHECK(i2c_driver_install(I2C_NUM_0, I2C_MODE_MASTER, 0, 0, 0)); // Init UART pour
SIM800L uart_config_t uart_conf = { .baud_rate = 9600, .data_bits = UART_DATA_8_BITS,
.parity = UART_PARITY_DISABLE, .stop_bits = UART_STOP_BITS_1, .flow_ctrl =
UART_HW_FLOWCTRL_DISABLE, }; ESP_ERROR_CHECK(uart_param_config(UART_NUM_1, &uart_conf));
ESP_ERROR_CHECK(uart_set_pin(UART_NUM_1, GPIO_SIM_TX, GPIO_SIM_RX, UART_PIN_NO_CHANGE,
UART_PIN_NO_CHANGE)); ESP_ERROR_CHECK(uart_driver_install(UART_NUM_1, 1024, 1024, 0, NULL,
0)); // Synchronisation inter-tasks state_events = xEventGroupCreate(); sensor_queue =
xQueueCreate(8, sizeof(sensor_reading_t)); prediction_queue = xQueueCreate(4,
sizeof(prediction_t)); // Lancer les 4 FreeRTOS tasks avec priorités adaptées
xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL); xTaskCreate(inference_task,
"inference", 8192, NULL, 4, NULL); // Plus de stack pour TFLite xTaskCreate(alarm_task,
"alarm", 4096, NULL, 3, NULL); xTaskCreate(sms_task, "sms", 4096, NULL, 6, NULL); //
Priorité élevée pour SMS ESP_LOGI(TAG, "All tasks started. VaxGuard is running."); // Le
main se termine ici, les tasks tournent en arrière-plan }
```

10.3. Partition table (partitions.csv)

Pour allouer suffisamment de flash au modèle TFLite (20 KB) tout en gardant de la place pour le firmware principal et les OTA updates, on définit une table de partitions personnalisée. Le modèle est stocké dans une partition data dédiée, ce qui permet de le mettre à jour indépendamment du firmware principal via OTA.

```
# Name, Type, SubType, Offset, Size, Flags nvs, data, nvs, , 0x4000, phy_init, data, phy, ,
0x1000, factory, app, factory, , 1M, ota_0, app, ota_0, , 1M, ota_1, app, ota_1, , 1M,
model, data, nvs, , 0x10000, # 64 KB pour le modèle TFLite
```

11. Firmware Option C — Edge Impulse (NoCode/LowCode)

Edge Impulse est une plateforme en ligne qui automatise une grande partie du pipeline : upload du dataset, configuration de l'architecture ("impulse"), entraînement via l'interface web, et téléchargement d'un firmware pré-compilé pour votre board. C'est l'option la plus rapide pour quelqu'un qui ne veut pas coder le firmware ni gérer la conversion ONNX → TFLite manuellement. En contrepartie, vous dépendez de la plateforme (limites du plan gratuit : 30 minutes de données d'entraînement max) et le modèle généré est moins optimisé que celui entraîné sur mesure.

11.1. Workflow étape par étape

Le workflow Edge Impulse comporte 5 étapes. Pour VaxGuard, nous utilisons le dataset CSV réel Bénin comme source d'entraînement, et nous configurons l'impulse comme un problème de time-series classification + régression (dual output : risk_class + minutes_to_threshold).

Étape	Action	Durée	Notes
1. Créer projet	Edge Impulse Studio → New project → "Time series data"	5 min	Nom: vaxguard-benin
2. Upload dataset	Upload le CSV download/data/vaxguard_real_dataset.csv	15 min	Format auto-detect
3. Design impulse	Input block: time-series (30×3) + Spectral Analysis	5 min	Window size 30, stride 5
4. Train	Classification (Neural Net) + Regression output → Train	20 min	20 epochs, validation 10%
5. Deploy	Deployment → ESP32-S3 DevKit → Download .zip	5 min	Firmware pré-compilé

Tableau 10 — Workflow Edge Impulse en 5 étapes.

11.2. Commandes CLI Edge Impulse

```
# 1. Lier la CLI à votre projet Edge Impulse edge-impulse-daemon # (Saisir les credentials si demandé) # La CLI va chercher votre projet "vaxguard-benin" # 2. Uploader le dataset CSV depuis votre PC edge-impulse-uploader --category training \
download/data/vaxguard_real_dataset.csv # 3. Après design + train via l'interface web, # télécharger le firmware compilé pour ESP32-S3 : edge-impulse-deployment download --target esp32s3-devkitc # 4. Flasher l'ESP32-S3 # (Décompressez le .zip téléchargé, puis) cd vaxguard-benin-esp32s3-firmware/ # Soit via Arduino IDE (ouvrir le .ino), # Soit via esptool : esptool.py --chip esp32s3 --port /dev/ttyUSB0 \ write_flash 0x0 firmware.bin # 5. Surveiller les logs esptool.py --chip esp32s3 --port /dev/ttyUSB0 \ monitor
```

12. Comparatif des 3 frameworks & recommandations

Maintenant que vous avez vu les 3 options en détail, voici un comparatif synthétique sur 10 critères pour vous aider à choisir. Pour le hackathon Small IA Banque Mondiale, le critère dominant est le temps de mise en route : la démo doit être prête en moins de 2 jours, sans risque de bug de dernière minute. C'est pourquoi Arduino est recommandé. Pour une suite post-hackathon (industrialisation au Bénin), ESP-IDF est le bon choix. Edge Impulse est pertinent si vous n'avez pas d'expertise en C++ embarqué.

Critère	Arduino + TFLite	ESP-IDF + TFLite	Edge Impulse
Difficulté	★★☆☆☆ Débutant	★★★★☆ Avancé	★☆☆☆☆ Très facile
Temps de dev	1-2 jours	1 semaine	< 1 jour

Critère	Arduino + TFLite	ESP-IDF + TFLite	Edge Impulse
Optimisation modèle	Bonne (TFLite INT8)	Excellente (custom)	Moyenne (auto)
RAM / Flash usage	Standard	Optimisé	Variable
OTA update	Difficile	✓ Natif	✓ Via plateforme
Multitâche real-time	SoftwareSerial	✓ FreeRTOS	Abstrait
Support communauté	Énorme	Bon	Grandissant
Coût	Gratuit	Gratuit	Gratuit (limité)
Flexibilité	Moyenne	Maximale	Faible
License	MIT	Apache 2	Propriétaire
Recommandé pour	Hackathon / Démo	Production	NoCode / Prototypage

Tableau 11 — Comparatif des 3 frameworks firmware.

Recommandation finale pour le hackathon : utilisez Arduino IDE + TFLite-Micro-ESP32. Le sketch de la section 9 est prêt à flasher et démontre toutes les fonctionnalités de VaxGuard (SHT31 + CNN-GRU + OLED + SMS GPS). Si vous gagnez le hackathon et voulez industrialiser, migrez vers ESP-IDF (section 10) pour gagner en robustesse.

13. Flasher le firmware sur l'ESP32-S3

Cette section décrit la procédure de flash pour chacun des 3 frameworks. Avant de commencer, vérifiez que l'ESP32-S3 est connecté via un câble USB-C (pas de câble de charge seul !) et que le port série est bien détecté par votre système. Sur Linux, vous devrez peut-être ajouter votre utilisateur au groupe dialout ou uucp pour avoir les droits d'accès au port /dev/ttyUSB0 ou /dev/ttyACM0.

13.1. Option Arduino IDE

```
# 1. Donner les droits port série (Linux uniquement) sudo usermod -a -G dialout $USER #
(déconnexion/reconnexion requise) # 2. Dans Arduino IDE : # Tools → Board → "ESP32S3 Dev
Module" # Tools → Port → /dev/ttyUSB0 (ou COM3 sur Windows) # Tools → Upload Speed → 921600
# 3. Ouvrir firmware_vaxguard.ino (section 9) # 4. Cliquer Upload (→) ou Ctrl+U # 5. Ouvrir
Serial Monitor (Ctrl+Shift+M) à 115200 baud # Vous devriez voir : # [0K] VaxGuard prêt #
[1/30] T=4.50°C H=65.20% # [2/30] T=4.52°C H=65.10% # ... # [30/30] T=5.20°C H=64.50% #
Inference: MTT=120.50 min, risk=0
```

13.2. Option ESP-IDF (CLI)

```
# 1. S'assurer que le path ESP-IDF est chargé . ~/esp/esp-idf/export.sh # 2. Aller dans le
projet cd /home/z/my-project/firmware/esp32 # 3. Configurer le target idf.py set-target
esp32s3 # 4. Configurer (optionnel – partition table custom) idf.py menuconfig # →
Partition Table → Custom partition table CSV # → Pointer vers partitions.csv (section 10.3)
# 5. Build idf.py build # 6. Flash + monitor (en une commande) idf.py -p /dev/ttyUSB0 -b
921600 flash monitor # Sortir du monitor : Ctrl + ]
```

13.3. Option Edge Impulse

```
# Après avoir téléchargé le .zip firmware depuis Edge Impulse (section 11) unzip
vaxguard-benin-esp32s3-firmware.zip cd vaxguard-benin-esp32s3-firmware # Soit via esptool
directement : pip install esptool esptool.py --chip esp32s3 --port /dev/ttyUSB0 \ --baud
921600 write_flash 0x0 firmware.bin # Soit via edge-impulse-daemon qui fait tout seul :
edge-impulse-daemon --target esp32s3-devkitc \ --port /dev/ttyUSB0 --firmware firmware.bin
```

13.4. Vérifier que le flash a réussi

Après flash, débranchez et rebranchez le câble USB-C pour forcer un redémarrage. L'OLED doit afficher "VaxGuard ready" avec la taille du tensor arena. Le moniteur série (à 115200 baud) doit afficher "[OK] VaxGuard prêt" dans les 5 secondes. Si vous voyez des caractères aléatoires ou des resets en boucle, c'est probablement un brownout : ajoutez un condensateur 470 µF sur le 3.3V et un 100 µF sur le 5V du SIM800L.

14. Tests & validation du prototype

Une fois le firmware flashé, il faut valider chaque sous-système indépendamment avant de tester le pipeline complet. Cette section décrit une procédure de test en 6 étapes, chacune avec un critère de succès clair. Comptez environ 1 heure pour tout valider.

14.1. Procédure de test par sous-système

Test	Procédure	Critère de succès	Durée
1. Capteur SHT31	Brancher, observer logs série	T° plausible 2-8°C en glacière, RH 50-70%	5 min
2. Display OLED	Vérifier allumage + affichage	Texte lisible, pas de flickering	2 min
3. Inférence CNN-GRU	Attendre warmup 30 steps (15 min sim)	MTT prédit 60-200 min en scenario normal	15 min
4. Alerte critique	Placer glacière au soleil 5 min	OLED affiche "CRITIQUE" + buzzer si présent	10 min
5. SMS GPS	Vérifier réception au dispensaire test	SMS reçu avec coords GPS + nom dispensaire	5 min
6. Autonomie batterie	Laisser tourner 4h sans USB	Batterie > 50% après 4h, pas de brownout	4h

Tableau 12 — Procédure de test complète du prototype VaxGuard.

14.2. Test spécifique scenario solar_burst

Pour démontrer la prédictivité du modèle au jury du hackathon, le scénario "solar_burst" est le plus visuel : on place la glacière dans un endroit ensoleillé pendant 5 minutes, et on observe l'IA passer progressivement de l'état "Sûr" (vert) à "Vigilance" (jaune) puis "Critique" (rouge) sur l'OLED, avant le déclenchement du SMS automatique. La température interne doit monter de 4.5°C à 8.5°C en environ 5 minutes réelles (équivalent à 10 minutes simulées), et le SMS doit être reçu par le dispensaire test dans les 30 secondes qui suivent le déclenchement.

***Astuce démo** : si vous n'avez pas de soleil le jour J, utilisez un sèche-cheveux à 30 cm de la glacière pendant 2 minutes pour simuler un solar_burst accéléré. Le modèle réagit de la même manière qu'au soleil réel, et la démo passe en 7 minutes au lieu de 15.*

15. Intégration avec le dashboard VaxGuard

Le dashboard Next.js (déployé sur GitHub Pages) affiche en temps réel les lectures capteurs, les prédictions du CNN-GRU, la carte GPS du Bénin, les SMS envoyés, les analyses Gemma 2B, l'historique des expéditions et la configuration. Pour connecter votre prototype physique ESP32-S3 au dashboard en temps réel, trois options sont possibles. Choisissez celle qui correspond à votre contexte de démo.

15.1. Option 1 : MQTT broker (production)

Pour une démo avec WiFi disponible, l'ESP32-S3 publie les readings sur un broker MQTT public (HiveMQ, Mosquitto), le dashboard Next.js s'abonne via websocket, et tout se met à jour en temps réel. Avantage : démo "vraiment" live. Inconvénient : dépend du WiFi, peu réaliste pour le Bénin rural.

15.2. Option 2 : SMS → parseur → dashboard

Pour respecter le principe "100% offline on vehicle", l'ESP32-S3 envoie un SMS au format JSON à un numéro routé vers un modem GSM branché au serveur. Le serveur FastAPI parse le SMS, met à jour l'état, et pousse le tout via WebSocket au dashboard. C'est l'option la plus fidèle à la vision produit de VaxGuard.

15.3. Option 3 : Mode standalone (recommandé hackathon)

Pour le hackathon, le prototype physique ESP32-S3 fonctionne en standalone (OLED + SMS GPS) sans connexion dashboard. Le dashboard affiche en parallèle les données du simulateur Python (qui tourne sur le PC du dispensaire), avec les analyses Gemma 2B. Les jurés voient donc les deux faces du produit : (1) le boîtier physique avec son OLED qui passe en CRITIQUE + SMS envoyé au dispensaire, et (2) le dashboard côté serveur qui montre la prédiction interprétée par le LLM en français.

***Astuce démo** : synchronisez manuellement le scénario simulé côté serveur avec le scénario réel côté ESP32 (lancez le "solar_burst" sur les deux en même temps). Les jurés verront l'IA physique (OLED) et l'IA serveur (dashboard) réagir en parallèle, ce qui est plus impactant que de tout simuler.*

16. Dépannage (Troubleshooting)

Cette section recense les 10 problèmes les plus fréquents rencontrés lors du montage et du flash du prototype VaxGuard, avec leur solution. La plupart des erreurs viennent de problèmes d'alimentation (brownout, alim insuffisante pour le SIM800L) ou de configuration I2C (adresses mal configurées). Si votre problème n'est pas listé, consultez le moniteur série Arduino à 115200 baud qui affiche des logs détaillés.

Problème	Cause probable	Solution
1. SHT31 non détecté	Adresse I2C 0x45 au lieu de 0x44	Scanner I2C (Wire scanner), vérifier pin ADDR
2. OLED reste noir	Contraste i2c addr 0x3D vs 0x3C	Modifier begin() avec addr correcte
3. SIM800L non init	Alim insuffisante (brownout)	Alim 5V 2A séparée + condensateur 470µF
4. Inférence trop lente	Modèle FP32 au lieu de INT8	Re-vérifier convert_model.py output
5. RAM insuffisante	Tensor arena trop petit	Augmenter kTensorArenaSize à 16 KB
6. OLED flickering	Bruit I2C, manque de découplage	Condensateur 100µF sur 3.3V + 10kΩ pull-ups
7. SMS non reçu	Carte SIM non activée, mauvais numéro	Tester SIM dans téléphone portable d'abord
8. Reset aléatoire	Brownout (alim 5V tire trop)	Condensateur 470µF sur 5V + alim 3A
9. Erreur TFLite "No matching op"	Ops custom non supportés	Recompiler tflite-micro avec all_ops_resolver
10. Modèle trop gros	Conversion mal faite	Re-quantifier en INT8 + shrink GRU hidden à 16

Tableau 13 — Top 10 problèmes rencontrés et leurs solutions.

17. Checklist finale avant présentation hackathon

Cette checklist doit être validée dans l'ordre avant votre passage devant le jury. Cochez chaque case mentalement ou sur papier. Si une case n'est pas cochée, ne présentez pas : corrigez d'abord. Cette discipline vous évitera les démos ratées qui font perdre des points précieux au hackathon.

17.1. Hardware

- Toutes les soudures sont propres (multimètre en mode continuité OK)
- Multimètre confirme 3.3V stable sur SHT31 et OLED
- Multimètre confirme 5V stable sur SIM800L (sous charge)
- Antenne GSM branchée et bien serrée sur le SIM800L
- Batterie LiPo chargée à 100% + câble USB-C de secours
- Glacière testée étanche (test eau 1h avant démo)
- Câble USB-C data (pas charge-only) pour flash
- Carte SIM testée (envoi SMS OK depuis un téléphone portable)

17.2. Firmware & modèle

- Dernier firmware compilé et flashé sans erreur

- model_data.h à jour (regénéré après convert_model.py)
- Scaler constants SCALER_MEAN et SCALER_SCALE correctes
- Logs série Arduino sans erreur ni warning (au moins 5 min de run)
- Partition table configurée (4 MB pour modèle, si ESP-IDF)
- Tensor arena size ≥ 8 KB (et pas d'erreur 'Arena size is insufficient')

17.3. Tests fonctionnels

- Test chauffeur normal : alerte non déclenchée en 15 min (OK)
- Test solar_burst : alerte CRITIQUE déclenchée en 5-10 min
- Test SMS : message reçu par dispensaire test (numéro +229XXXXXXXX)
- Test OLED : texte lisible, mise à jour toutes les 30s
- Autonomie batterie > 4h sans USB branché
- Pas de reset aléatoire en 30 min de run continu

17.4. Démo & pitch

- Dashboard GitHub Pages accessible (<https://kajarnaklokossou2008.github.io/vaxguard/>)
- Screenshots dashboard en backup (si coupure réseau le jour J)
- Pitch 90 secondes préparé (problème → solution → impact)
- Q&A; anticipée (prix, scalabilité, industrialisation Bénin, données réelles)
- Repo GitHub privé/public accessible au jury
- Modèle 3D ou photo du prototype imprimée (impact visuel)
- Backup batteries (2 LiPo chargées) + câble USB-C de rechange
- Scénario solar_burst préparé (sèche-cheveux ou placement soleil)

17.5. Message motivation

Pourquoi ce projet compte : chaque année, 50% des vaccins sont gaspillés à cause de la rupture de la chaîne du froid. Au Bénin, ce gaspillage prend un visage humain : des enfants reçoivent des vaccins inefficaces, des patients meurent d'insuline dégradée. VaxGuard, c'est 17.2 KB d'Edge-AI qui peuvent sauver des vies — pas juste gagner un hackathon. Présentez avec cette conviction : vous avez construit quelque chose qui peut vraiment déployer au Bénin, à 80€ le boîtier, sans dépendre d'Internet.

18. Ressources & références

Cette section liste les ressources externes qui vous seront utiles pour aller plus loin : documentation officielle des composants, tutoriels, datasets, communautés. Conservez ces liens : ils vous serviront aussi bien pendant le hackathon que pour l'industrialisation post-hackathon au Bénin.

18.1. Documentation officielle

- **ESP32-S3 Datasheet** — Espressif Systems. PDF 1500+ pages.
https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- **TFLite-Micro-ESP32** — GitHub repository. <https://github.com/tanakamasayuki/tensorflowlite-esp32>
- **TensorFlow Lite Micro** — Documentation officielle. <https://www.tensorflow.org/lite/microcontrollers>
- **Edge Impulse Docs** — <https://docs.edgeimpulse.com/>
- **Adafruit SHT31 Library** — https://github.com/adafruit/Adafruit_SHT31
- **Adafruit SSD1306** — https://github.com/adafruit/Adafruit_SSD1306
- **SIM800L AT Commands Manual** — PDF 200+ pages, recherche Google.
- **ESP-IDF Programming Guide** — <https://docs.espressif.com/projects/esp-idf/>

18.2. Tutoriels & blogs

- **Random Nerd Tutorials** — ESP32 + sensors + projects. <https://randomnerdtutorials.com/projects-esp32/>
- **Edge Impulse Blog** — Tutorials + case studies. <https://www.edgeimpulse.com/blog>
- **Hackster.io** — ESP32 TinyML projects. <https://www.hackster.io/edge-ml>
- **Adafruit Learn** — SHT31 + OLED wiring. <https://learn.adafruit.com/>
- **PyTorch Quantization** — <https://pytorch.org/docs/stable/quantization.html>
- **onnx2tf GitHub** — <https://github.com/PINTO0309/onnx2tf>

18.3. Dataset & modèle VaxGuard

- **Open-Meteo Archive API** — Données météo historiques gratuites.
<https://open-meteo.com/en/docs/historical-weather-api>
- **Repo GitHub VaxGuard** — <https://github.com/KajarnakLOKOSSOU2008/vaxguard>
- **Modèle entraîné** — download/models/vaxguard_cnn_gru_real_int8.pt (17.2 KB INT8)
- **Dataset réel Bénin** — download/data/benin_abomey_real_weather.json (1464 heures)
- **Manifest JSON** — download/models/model_manifest_real.json (métriques + chemins)
- **Dashboard live** — <https://kajarnaklokossou2008.github.io/vaxguard/>

18.4. Communautés & support

- **Edge Impulse Discord** — Communauté active pour TinyML. ~3000 membres.
- **Reddit r/esp32** — Communauté ESP32 (~80k membres).
- **Reddit r/MachineLearning** — Pour discussions TinyML.
- **Stack Overflow tags** : esp32, esp-idf, tensorflow-lite, onnx
- **Espressif Forum** — Support officiel ESP-IDF. <https://forum.esp32.com/>
- **HuggingFace Forums** — Pour Gemma 2B + GGUF.

18.5. Pour aller plus loin

- **Quantization-Aware Training (QAT)** — <https://pytorch.org/docs/stable/quantization.html#quantization-aware-training>
- **MicroTorch** — Runtime PyTorch minimaliste pour microcontrôleurs. <https://github.com/cansik/micro-torch>
- **ESP-DL** — Framework Deep Learning officiel Espressif. <https://github.com/espressif/esp-dl>
- **Gemma 2B Model Card** — <https://huggingface.co/google/gemma-2-2b-it>
- **llama-cpp-python** — <https://github.com/abetlen/llama-cpp-python>
- **GitHub Actions for Pages** — <https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages>

18.6. Licence & crédits

Le code VaxGuard est open source (licence MIT), librement réutilisable et modifiable. Le modèle Gemma 2B est mis à disposition par Google sous la licence Gemma Terms of Use. Les données météo Open-Meteo sont sous licence CC BY 4.0 (attribution requise). Les librairies Adafruit, ESP-IDF et TensorFlow sont sous leurs licences respectives (MIT, Apache 2, Apache 2). Ce guide a été rédigé par Dossou Kajarnak LOKOSSOU pour le Hackathon Small IA de la Banque Mondiale, octobre 2026.

Fin du guide. Pour toute question, ouvrir une issue sur le repo GitHub ou contacter l'auteur directement. Bonne chance pour le hackathon !